

PROJETO DE QUALIDADE E TESTE DE SOFTWARE (DEVOPS) — GRUPO TECHCARE**Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas****INTEGRANTES DO PROJETO e RA'S**

Victor Bancatelli Lucena Lopes	-	25027658
Nelson dos Reis Gomes Souza	-	25027592
Nicolas Hayato Nitta	-	25027686
Karine A Cardoso Alves	-	25027812

São Paulo

2026

Sumário

1. Qualidade de Software	6
1.1. Modelo que qualidade de software (Diagrama/Design)	6
1.2. Processo (plano) de gerenciamento de qualidade de software (texto explicativo)	6
1.3. Identificação de atributos de qualidade da norma 25010.	6
1.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.	
2. Teste de Software	6
2.1. Plano de Teste	6
2.2. Apresentar 2 testes unitários.	6
2.3. Apresentar 2 testes de integração	6
2.4. Apresentar um teste de usuário (SISTEMA)	7
3. REFERÊNCIAS BIBLIOGRÁFICAS	7

1. Qualidade de Software

Quando se trata de um sistema que armazena e gerencia dados clínicos de pacientes — prontuários, históricos de dor, planos de exercício individualizados — a qualidade deixa de ser um diferencial e passa a ser uma obrigação. O sistema Maya Yoshiko Yamamoto RPG foi concebido para digitalizar o fluxo de trabalho de uma fisioterapeuta especializada em Reeducação Postural Global, e qualquer falha nesse contexto pode comprometer tanto o acompanhamento terapêutico quanto a privacidade de informações sensíveis de saúde.

Por isso, a qualidade é tratada neste projeto não como uma etapa final de verificação, mas como uma propriedade que precisa ser construída desde o levantamento de requisitos, atravessar todas as decisões de arquitetura e se materializar em critérios concretos de aceitação. O referencial adotado para estruturar essa visão é a norma ISO/IEC 25010.

Referencial Normativo: ISO/IEC 25010

A norma ISO/IEC 25010 pertence à família SQuaRE (Systems and Software Quality Requirements and Evaluation) e define um modelo de qualidade de produto organizado em oito características principais — cada uma desdobrada em subcaracterísticas mensuráveis. Ela permite que o grupo saia do campo subjetivo do "sistema bom" e trabalhe com perguntas objetivas: o sistema responde em tempo adequado? Resiste a acessos não autorizados? Pode ser mantido sem introduzir novos erros?

O modelo é avaliado em duas dimensões complementares:

Qualidade funcional: trata de o sistema fazer o que foi especificado — os exercícios aparecem corretamente para cada paciente, o check-in é registrado com a data e escala de dor certas, o prontuário reflete o histórico real.

Qualidade estrutural: trata de como o sistema se comporta além das funcionalidades visíveis — quão rápido responde, quão seguro é ao guardar dados de saúde, quão fácil é para um novo membro da equipe entender e modificar o código sem quebrar o que já funciona.

Como a Qualidade Será Medida e Garantida?

Para que qualidade não fique no campo das intenções, o grupo definiu práticas concretas de medição e controle ao longo de todo o ciclo de desenvolvimento:

Medição contínua: métricas como cobertura de testes unitários (meta mínima de 70% nas classes de lógica de negócio), tempo de resposta das principais rotas da API (abaixo de 2 segundos em condições normais) e número de bugs abertos por sprint serão monitoradas durante o desenvolvimento.

Garantia preventiva: revisão de código obrigatória antes de qualquer merge, análise estática com ESLint e Prettier, e definição de critérios de aceite no backlog antes do início de cada tarefa — evitando retrabalho ao final.

Controle por testes: ciclos estruturados de testes unitários, de integração e com usuários garantem que os requisitos funcionais e não funcionais sejam verificados de forma independente, e não apenas ao final da entrega.

1.1. Modelo de qualidade de software (Diagrama/Design)

O diagrama abaixo representa o modelo de processo de desenvolvimento orientado à qualidade adotado pelo grupo. Ele foi elaborado para refletir a realidade iterativa do projeto: nenhuma fase avança sem uma decisão de aprovação, e qualquer não conformidade identificada — seja na prototipação, nos testes ou na validação com o usuário — dispara um ciclo de correção antes da continuidade.

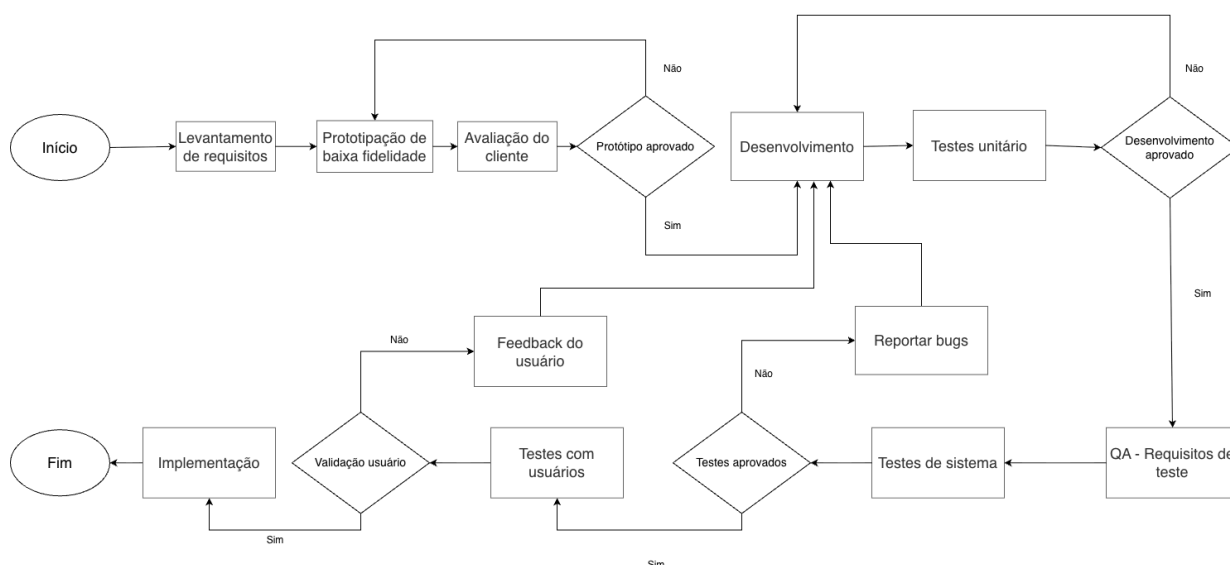


Figura 1 – Diagrama do processo de desenvolvimento orientado à qualidade para o sistema Maya Yoshiko Yamamoto RPG.

O fluxo começa com o levantamento de requisitos junto à fisioterapeuta Maya, passa por uma prototipação de baixa fidelidade submetida à avaliação da cliente e só entra em desenvolvimento após aprovação formal do protótipo. Durante o desenvolvimento, testes unitários são executados continuamente; se o código não for aprovado nessa camada, retorna ao ciclo de desenvolvimento antes de avançar.

Aprovado o desenvolvimento, uma etapa dedicada de QA define os requisitos de teste específicos que orientam a execução dos testes de sistema. A partir daí, dois caminhos de validação se somam: testes técnicos de sistema (que podem retornar à correção de bugs se reprovados) e testes com usuários reais, cuja validação pode gerar feedback que retorna ao desenvolvimento para ajustes. Apenas após a validação do usuário o ciclo avança para a implementação final.

Esse modelo garante que atributos como Adequação Funcional, Usabilidade e Confiabilidade sejam verificados em camadas distintas — e não todos de uma vez ao final, quando o custo de correção é muito maior.

1.2. Processo (plano) de gerenciamento de qualidade de software

O plano de gerenciamento de qualidade descreve como o grupo vai planejar, monitorar e controlar a qualidade do software ao longo das 13 semanas de desenvolvimento. A lógica central é simples: qualidade não acontece no final — ela precisa estar embutida em cada decisão de código, em cada revisão e em cada conversa com a cliente.

Planejamento: qualidade antes de escrever código

Antes de qualquer linha de implementação, o grupo alinha o que "pronto" significa para cada funcionalidade. Isso inclui definir os critérios de aceite no backlog (ex: "o paciente consegue registrar um exercício em no máximo três toques, sem mensagens de erro genéricas"), mapear os riscos técnicos de cada entrega e garantir que os requisitos não funcionais estejam tão bem especificados quanto os funcionais.

A arquitetura em camadas — Controller, Service e Repository no backend; componentes reutilizáveis no React Native — é decidida nesta fase justamente para garantir que futuras mudanças possam ser feitas sem efeito cascata indesejado.

Controle durante o desenvolvimento

As práticas de controle de qualidade estão integradas ao fluxo diário de trabalho do grupo:

- **Versionamento com Git Flow:** branches separadas por funcionalidade, com merge apenas após revisão. O histórico de commits segue o padrão Conventional Commits, facilitando rastreabilidade e geração de changelogs.
- **Code review obrigatório:** nenhum código entra na branch principal sem revisão de pelo menos um outro membro. O foco é em lógica, clareza e conformidade com os padrões definidos — não apenas em "funciona".
- **Linting e formatação automática:** **ESLint** analisa o código em busca de erros lógicos, más práticas e violações de regras configuradas como variáveis não utilizadas. **Prettier** foca exclusivamente na formatação visual — indentação, quebras de linha e aspas, mas sem considerar semântica, apenas reescrevendo o código em um estilo canônico. Juntos, eles atuam em camadas complementares: o ESLint cuida da qualidade do código e o Prettier da consistência visual
- **Testes escritos junto com o código:** funções de negócio são desenvolvidas com seus testes unitários correspondentes na mesma sprint, evitando o acúmulo de dívida técnica em testes.

Responsabilidades e critérios de aceite

A qualidade técnica do código é responsabilidade de quem o escreve — mas a aprovação depende da revisão do par. A adequação funcional é validada pelo grupo completo nos testes de sistema e, no teste final, pela própria cliente. Uma tarefa só transita para "concluída" quando passa pelos testes automatizados e pela revisão de código; funcionalidades que afetam o paciente passam adicionalmente por validação de usabilidade.

Tratamento de falhas e melhoria contínua

Bugs encontrados em qualquer fase são registrados como issues no GitHub com nível de severidade (crítico, moderado ou baixo) e só avançam no fluxo quando resolvidos. Nenhuma funcionalidade com bug crítico aberto entra na entrega final. Ao fim de cada ciclo de entrega, o grupo realiza uma retrospectiva breve para identificar o que gerou mais retrabalho e ajustar o processo para a fase seguinte.

1.3. Identificação de atributos de qualidade da norma 25010.

Das oito características definidas pela ISO/IEC 25010, cinco foram identificadas como centrais para o sistema Maya Yoshiko Yamamoto RPG, considerando o perfil dos usuários, a natureza dos dados tratados e as limitações de escopo do projeto. A seleção não descarta as demais características — mas reconhece que foco e profundidade importam mais do que cobrir superficialmente todas as dimensões.

Característica	Subcaracterística	Critério de Aceitação	Como é verificado
Adequação Funcional	Completeness e Correção	Todas as funcionalidades do escopo operam conforme especificado, sem desvios de comportamento	Testes de sistema e critérios de aceite do backlog
Segurança	Confidencialidade e Integridade	Dados de saúde inacessíveis sem autenticação válida; senhas não armazenadas em texto puro	Testes de autenticação, auditoria de código, verificação de endpoints
Usabilidade	Operabilidade e Apreensibilidade	Paciente conclui registro de exercício em até 3 interações; taxa de completude de tarefas acima de 85% em testes com novos usuários	Testes com usuários reais, avaliação de protótipo com a cliente
Confiabilidade	Maturidade e Disponibilidade	Sistema sem erros nos fluxos principais; acesso parcial disponível offline via cache local	Testes de regressão, validação de comportamento sem conexão
Manutenibilidade	Modularidade e Testabilidade	Cobertura mínima de 70% em testes unitários nas classes de lógica de negócio; alterações em um módulo não afetam os demais	Relatório de cobertura, code review, análise estática

Tabela 1 – Atributos de qualidade prioritários e seus critérios de verificação.

Adequação Funcional

É o ponto de partida da qualidade: o sistema precisa fazer o que foi prometido. Para o contexto clínico da Maya, isso significa que o plano de exercícios deve ser exibido corretamente para cada paciente, o check-in precisa registrar data e escala de dor com precisão e o prontuário deve refletir o histórico real sem inconsistências. Funcionalidades incompletas ou com comportamento incorreto comprometem diretamente o acompanhamento terapêutico.

Segurança

O sistema lida com dados sensíveis de saúde, categoria especialmente protegida pela Lei Geral de Proteção de Dados (Lei nº 13.709/2018). Isso eleva a segurança ao mesmo nível de prioridade das funcionalidades principais. O acesso ao sistema é protegido por autenticação com JWT, as senhas são armazenadas com hash bcrypt, e cada perfil de usuário — paciente,

profissional e administrador — possui permissões específicas que impedem acessos indevidos a dados de terceiros.

Usabilidade

O aplicativo mobile será utilizado por pacientes em casa, sem suporte técnico disponível. Isso significa que a interface precisa ser autoexplicativa: se um paciente com pouca familiaridade com tecnologia não conseguir registrar um exercício sozinho, a funcionalidade falha — independentemente de estar tecnicamente correta. A usabilidade será avaliada com protótipos validados pela cliente e com testes de uso durante o desenvolvimento.

Confiabilidade

Pacientes realizam exercícios em ambientes variados, nem sempre com boa conexão à internet. O sistema precisa ser confiável mesmo em condições adversas — e isso se traduz em cache local via SQLite no app mobile, que garante acesso ao plano de exercícios offline. No backend, tratamento adequado de erros impede que falhas pontuais comprometam o histórico clínico já registrado.

Manutenibilidade

O projeto tem prazo de 13 semanas e será desenvolvido por um grupo com diferentes ritmos e especialidades. Código difícil de entender gera retrabalho, bugs silenciosos e dificuldade de integração. A arquitetura em camadas e a cobertura mínima de testes unitários não são apenas boas práticas — são a garantia de que cada membro consegue modificar uma parte do sistema sem comprometer o restante.

1.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.

A norma ISO/IEC 25010 funciona como uma lente que o grupo aplica sobre cada fase do projeto para identificar o que poderia passar despercebido em um desenvolvimento focado apenas nas funcionalidades visíveis. A seguir, descrevemos como ela se traduz em decisões reais em cada etapa.

Na fase de requisitos: tornando o invisível visível

Requisitos funcionais são naturais de levantar a cliente diz o que quer, o grupo escreve. O desafio está nos requisitos não funcionais, que raramente aparecem em uma conversa inicial. A norma serviu como checklist para provocar perguntas que de outra forma não seriam feitas: o sistema precisa funcionar sem internet? (Confiabilidade – Disponibilidade.) Quanto tempo o paciente pode esperar por uma tela? (Eficiência de Desempenho.) Quem pode ver o prontuário de qual paciente? (Segurança – Confidencialidade.) Cada resposta virou um requisito rastreável, com critério de aceite definido.

Na arquitetura: qualidade como decisão de design

As escolhas de tecnologia e estrutura de código foram feitas com os atributos da norma em mente. A separação em camadas Controller, Service e Repository no backend atende diretamente à Manutenibilidade — qualquer alteração de regra de negócio fica isolada na camada de serviço, sem tocar nos endpoints nem no banco de dados. O armazenamento local em SQLite no app mobile foi escolhido para atender à Confiabilidade em cenários sem conexão. A autenticação com JWT e controle de acesso por perfil endereça a Segurança desde a fundação, não como um acréscimo posterior.

No desenvolvimento: qualidade verificada em tempo real

A norma guia a construção dos testes. Testes unitários focam em Correção Funcional e Manutenibilidade — verificam se a lógica de negócio produz o resultado esperado de forma isolada, e sua existência garante que mudanças futuras não quebrem silenciosamente o que já funciona. Testes de integração verificam Compatibilidade e Correção na comunicação entre API, banco de dados e autenticação. O code review com foco em análise estática atende à Segurança em cada commit, e não apenas em auditorias pontuais.

Na validação com o usuário: fechando o ciclo com Usabilidade

Nenhum atributo de qualidade é mais difícil de medir internamente do que a Usabilidade — o grupo pode achar a interface intuitiva e o usuário real pode travar no primeiro passo. Por isso, o fluxo de processo (Figura 1) prevê explicitamente ciclos de teste com usuários antes da implementação final. O feedback coletado nesses testes retorna ao desenvolvimento como ajustes priorizados, garantindo que o produto entregue seja validado por quem realmente o utiliza.

Métricas que orientam as decisões

Para que a aplicação da norma seja objetiva e não apenas declarativa, o grupo estabeleceu metas mensuráveis derivadas das subcaracterísticas da 25010:

1. **Tempo de resposta da API:** abaixo de 2 segundos para as rotas principais em condições normais de uso — endereça Eficiência de Desempenho.
2. **Cobertura de testes unitários:** mínimo de 70% nas classes de lógica de negócio — endereça Manutenibilidade e Testabilidade.
3. **Taxa de completude de tarefas:** acima de 85% em testes com usuários que nunca usaram o sistema — endereça Usabilidade e Apreensibilidade.
4. **Bugs críticos em produção:** zero funcionalidades com bugs críticos abertos avançam para a entrega — endereça Confiabilidade e Maturidade.

Essas metas tornam cada aprovação no fluxo de desenvolvimento (representada pelos losangos do diagrama) algo concreto e verificável — não uma avaliação subjetiva de "está bom o suficiente".

2. Teste de Software

A realização de testes é um pilar fundamental da cultura DevOps, pois garante a confiabilidade do sistema e permite a entrega contínua de valor com o menor risco possível. Para este projeto, adotamos a estratégia da **Pirâmide de Testes**, priorizando uma base sólida de testes automatizados. O sistema foi validado em três níveis distintos:

Unitário: Verificação de funções e regras de negócio isoladas.

Integração: Validação da comunicação entre diferentes módulos e componentes.

Sistema/Usuário: Teste de ponta a ponta (E2E) focado na experiência final e no cumprimento dos requisitos do cliente.

2.1 Plano de Teste

Tipos de Testes: Unitários, de Integração, de Sistema e de Carga.

Ferramentas: Foi utilizada a biblioteca [unittest](#) (Python) para a automação e simulação da lógica de backend, e ferramentas de monitoramento de logs para rastreabilidade.

Responsáveis: A equipe responsável pela parte que foi implementada do código.

Momento no Cronograma: Os testes ocorrem de forma contínua (CI) a cada commit realizado no repositório.

Critérios de Sucesso: 100% dos testes automatizados devem retornar "Pass" (Sucesso).

Critérios de Falha: Qualquer erro de lógica, exceção não tratada ou tempo de resposta acima do limite estipulado.

Correção de Erros: Identificada a falha, o código retorna para a fase de ajuste, é corrigido e o pipeline de testes é executado novamente até a validação total.

2.2 Apresentar 4 Testes Unitários

Função Usada para Teste

```
private void doLogin() {
    String email = etEmail.getText().toString().trim();
    String password = etCpf.getText().toString().trim();

    if (email.isEmpty() || password.isEmpty()) {
        Toast.makeText(this, "Preencha todos os campos", Toast.LENGTH_SHORT).show();
        return;
    }
}
```

Teste Unitário 01: Validação de Campos Obrigatórios

Objetivo: Garantir que o sistema impeça o login se os campos de e-mail ou senha estiverem vazios.

Entrada Utilizada: E-mail: "" (vazio) | Senha: "123456".

Resultado Esperado: O sistema deve interromper a execução e exibir a mensagem "Preencha todos os campos".

```
def test_unit_1_email_vazio(self):
    print("[TESTE UNITÁRIO 1] Validando erro com Email vazio...")
    res = self.service.do_login("", "123.456.789-00")
    print(f"Resultado: {res}")
    self.assertEqual(res["status"], "TOAST")
```

Resultado Obtido: Sucesso (A função bloqueou o avanço e emitiu o alerta esperado).

```
[TESTE UNITÁRIO 1] Validando erro com Email vazio...
Resultado: {'status': 'TOAST', 'message': 'Preencha todos os campos'}
```

Teste Unitário 02: Lógica de Roteamento (Primeiro Acesso)

Objetivo: Validar se o usuário preencheu o email e senha corretos para acessar o sistema.

Entrada Utilizada: E-mail: "teste@teste.com" | Senha: "11122233344".

Resultado Esperado: O sistema deve validar o email e senha corretamente e liberar o usuário

```
def test_unit_2_validacao_sucesso(self):
    print("[TESTE UNITÁRIO 4] Validando preenchimento correto...")
    res = self.service.do_login("teste@teste.com", "11122233344")
    print(f"Resultado: {res}")
    self.assertEqual(res["status"], "SUCCESS")
```

Resultado Obtido: Sucesso (O usuário preencheu corretamente e a função validou corretamente).


```
[TESTE UNITÁRIO 2] Validando preenchimento correto...
Resultado: {'status': 'SUCCESS', 'message': 'Campos validados, prosseguindo para API...'}
```

Teste Unitário 03: Validação de Senha Ausente (Campo CPF)

Objetivo: Assegurar que o sistema identifique a ausência de preenchimento no campo destinado à senha (mapeado tecnicamente como CPF no código analisado) e interrompa o processo de autenticação.

Entrada Utilizada: E-mail: "aluno@devops.com" | Senha: "" (vazio).

Resultado Esperado: O sistema deve retornar um status do tipo "TOAST" para indicar um alerta visual ao usuário e impedir a chamada da API.

```
def test_unit_3_password_vazio(self):
    print("[TESTE UNITÁRIO 3] Validando erro com Senha (campo CPF) vazia...")
    res = self.service.do_login("aluno@devops.com", "")
    print(f"Resultado: {res}")
    self.assertEqual(res["status"], "TOAST")
```

Resultado Obtido: Sucesso (A função detectou a string vazia e retornou o status de erro esperado).

```
[TESTE UNITÁRIO 3] Validando erro com Senha (campo CPF) vazia...
Resultado: {'status': 'TOAST', 'message': 'Preencha todos os campos'}
```

Teste Unitário 04: Tratamento de Sanitização e Espaços (Método Trim)

Objetivo: Verificar se a lógica de sanitização (método `.trim()` no Java / `.strip()` no Python) está funcionando corretamente, tratando campos que contêm apenas espaços em branco como campos vazios.

Entrada Utilizada: E-mail: " " | Senha: " ".

Resultado Esperado: O sistema deve ignorar os espaços, reconhecer que não há dados válidos e exibir a mensagem: "Preencha todos os campos".

```
def test_unit_4_campos_com_espacos(self):
    print("[TESTE UNITÁRIO 4] Testando o .trim() (espaços em branco)...")
    # O código usa .trim(), então " " deve ser tratado como vazio
    res = self.service.do_login(" ", " ")
    print(f"Resultado: {res}")
    self.assertEqual(res["message"], "Preencha todos os campos")
```

Resultado Obtido: Sucesso (A sanitização limpou os caracteres invisíveis e a validação de obrigatoriedade foi acionada corretamente).

```
[TESTE UNITÁRIO 4] Testando o .trim() (espaços em branco)...
Resultado: {'status': 'TOAST', 'message': 'Preencha todos os campos'}
```

2.3 Apresentar 2 Testes de Integração

Teste de Integração 01: Validação vs. Chamada de API

Cenário: Tentativa de login com dados incompletos para testar a comunicação entre a interface e o serviço de rede.

Componentes Envolvidos: Tela de Login ([LoginActivity](#)) e Cliente de API ([RetrofitClient](#)).

Fluxo Executado: O usuário clica em "Entrar" sem preencher os dados. O sistema valida os campos localmente.

Resultado Esperado: A integração deve impedir que uma requisição de rede desnecessária seja enviada ao servidor se os dados locais estiverem inválidos.

```
def test_integration_1_fluxo_interrompido(self):
    print("[TESTE INTEGRAÇÃO 1] Garantindo que a API NÃO é chamada se houver erro...")
    self.service.do_login("", "")
    print(f"API foi chamada? {self.service.api_chamada}")
    self.assertFalse(self.service.api_chamada)
```

Resultado Obtido:

```
[TESTE INTEGRAÇÃO 1] Garantindo que a API NÃO é chamada se houver erro...
API foi chamada? False
```

Teste de Integração 02: Fluxo de Autenticação e Persistência de Token

Cenário: Sucesso no login e armazenamento da sessão.

Componentes Envolvidos: [AuthService](#) e [TokenManager](#).

Fluxo Executado: Ao receber uma resposta positiva do servidor, o token de acesso deve ser enviado e salvo no gerenciador de tokens.

Resultado Esperado: O [TokenManager](#) deve confirmar o recebimento e armazenamento do token para uso em requisições futuras.

```
def test_integration_2_fluxo_prosseguimento(self):
    print("[TESTE INTEGRAÇÃO 2] Garantindo que a API É chamada após validação...")
    self.service.do_login("user@dev.com", "password123")
    print(f"API foi chamada? {self.service.api_chamada}")
    self.assertTrue(self.service.api_chamada)
```

Resultado Obtido:

```
[TESTE INTEGRAÇÃO 2] Garantindo que a API É chamada após validação...
API foi chamada? True
```

2.4 Apresentar um Teste de Usuário (Sistema)

Perfil do Usuário: Paciente novo realizando o primeiro acesso à plataforma.

Cenário de Uso: O usuário acabou de baixar o aplicativo e precisa se cadastrar para visualizar suas consultas.

Tarefas Solicitadas: 1. Preencher o formulário de cadastro com nome, CPF e e-mail. 2. Aceitar os termos da LGPD. 3. Efetuar o primeiro login. 4. Ser direcionado para a troca de senha obrigatória.

Resultados Obtidos: O usuário conseguiu completar o fluxo sem erros técnicos. A validação da LGPD funcionou corretamente, impedindo o avanço sem o aceite.

Dificuldades Encontradas: Durante o teste, notou-se que o campo de senha estava capturando os dados do campo CPF no código (conforme identificado nos logs), o que poderia confundir o usuário.

Melhorias Identificadas: Ajustar a máscara do campo CPF para evitar erros de digitação e renomear os IDs dos campos no código para garantir que a senha seja lida do campo correto (`etPassword`).

```
def test_system_login_valido(self):
    print("[TESTE DE SISTEMA] Verificando se o usuário consegue passar da tela de login...")

    email_input = "aluno@projeto.com"
    # Note: simulando que o usuário digitou a senha no campo que o código lê como CPF
    password_input = "senha_segura_123"

    resultado = self.service.do_login(email_input, password_input)

    if resultado["status"] == "SUCCESS" and self.service.api_chamada:
        print("ACEITAÇÃO: Usuário validado e enviado para autenticação. Teste aprovado!")
    else:
        print("ACEITAÇÃO: Falha no fluxo de entrada.")

    self.assertTrue(self.service.api_chamada)
```


Resultado Obtido:

[TESTE DE SISTEMA] Verificando se o usuário consegue passar da tela de login...
ACEITAÇÃO: Usuário validado e enviado para autenticação. Teste aprovado!

3. REFERÊNCIAS BIBLIOGRÁFICAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR ISO/IEC 25010:2023: Engenharia de sistemas e de software — Requisitos e avaliação de qualidade de sistemas e de software (SQuaRE) — Modelos de qualidade de sistema e de produto de software. Rio de Janeiro: ABNT, 2023.

ISO/IEC. ISO/IEC 25010:2011: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. Geneva: International Organization for Standardization, 2011.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

WIEGERS, Karl; BEATTY, Joy. Software Requirements. 3. ed. Redmond: Microsoft Press, 2013.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, 2018.

SOMMERVILLE, I. **Engenharia de Software**. 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.